

Module : Système d'exploitation 2
MIP / S4
Département Mathématiques et Informatique

TD 4 : Système de gestion de fichiers

Exercice 1 : File Allocation Table (FAT)

Le schéma de la **figure 1** représente l'allocation d'un disque Windows formaté avec un système de gestion de fichiers (SGF) de type FAT. Les blocs sont numérotés de **1 à 20**, ligne par ligne, de gauche à droite.

1. Représentez les 20 premières entrées de la table d'allocation.

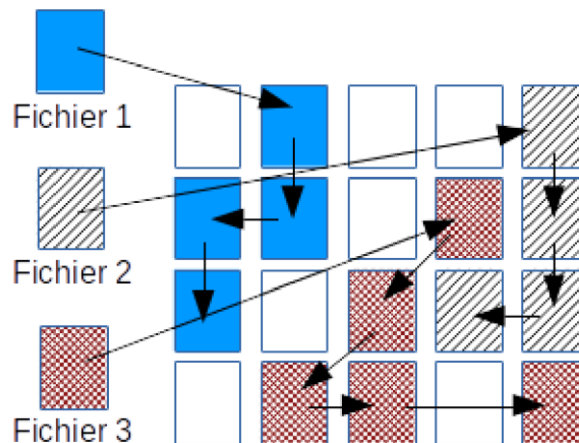


figure : allocation d'un disque Windows avec un SGF de type FAT

2. Le nom FAT16 signifie que les numéros de bloc sont écrits sur 16 bits. Si on suppose qu'un bloc fait 32 Ko, quelle est la taille maximale adressable ?
3. FAT32 est la dernière version de FAT. Les numéros de blocs sont codés sur 32 bits (en réalité 28 bits).
Quelle est la taille maximale adressable ?
Microsoft a toutefois limité cette capacité à 32 Go pour favoriser l'usage de NTFS.

1. Représentez les 20 premières entrées de la table d'allocation

Bloc	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Valeur de la FAT	Libre	7	Libre	Libre	10	11	6	Libre	13	15	NULL	Libre	17	NULL	14	Libre	18	20	Libre	NULL

2. Le nom FAT16 signifie que les numéros de bloc sont écrits sur 16 bits. Si on suppose qu'un bloc fait 32 Ko, quelle est la taille maximale adressable ?.

- Étape 1 : FAT = File Allocation Table (table d'allocation des fichiers).
16 signifie que chaque numéro de bloc est codé sur 16 bits.

$$2^{16}=65536 \text{ numéros de blocs possibles.}$$

- Étape 2 : 1 bloc = 32 Ko (kilo-octets).
 $32\text{Ko}=32\times 1024=32768 \text{ octets par bloc.}$

- Étape 3 : Calcul de la capacité maximale adressable
Si on a 65536 blocs et que chaque bloc fait 32 Ko :

$$65536\times 32768=2147483648 \text{ octets}$$

Convertissons en unités plus lisibles :

$$2147483648 \text{ octets} = 2147483648 \div 1024 = 2097152 \text{ Ko}$$

$$2097152 \div 1024 = 2048 \text{ Mo}$$

$$2048 \div 1024 = 2 \text{ Go.}$$

Réponse : Taille maximale adressable avec FAT16 = 2 Go

3. FAT32 est la dernière version de FAT. Les numéros de blocs sont codés sur 32 bits (en réalité 28 bits).

- Étape 1 : FAT = File Allocation Table (table d'allocation des fichiers).
32 signifie que chaque numéro de bloc est codé sur 28 bits.

$$2^{28}=268435456 \text{ numéros de blocs possibles.}$$

- Étape 2 : 1 bloc = 32 Ko (kilo-octets).
 $32\text{Ko}=32\times 1024=32768 \text{ octets par bloc.}$

- Étape 3 : Calcul de la capacité maximale adressable
Si on a 268435456 blocs et que chaque bloc fait 32 Ko :

$$268435456\times 32768=8796093022208 \text{ octets}$$

Convertissons en unités plus lisibles :

$$8796093022208 \text{ octets} = 8796093022208 \div 1024 = 8589934592 \text{ Ko}$$

$$8589934592 \div 1024 = 8388608 \text{ Mo}$$

$$8388608 \div 1024 = 8192 \text{ GB.}$$

$$8192 \div 1024 = 8 \text{ TB.}$$

Réponse : Taille maximale adressable avec FAT32 = 8 TB

Principe général

Pour calculer la capacité maximale adressable d'un système de fichiers FAT :

1. Étape 1 : **Déterminer le nombre maximum de blocs :**
 - FAT12 $\rightarrow 2^{12} = 4096$ blocs
 - FAT16 $\rightarrow 2^{16} = 65536$ blocs
 - FAT32 $\rightarrow 2^{28} = 268\,435\,456$ blocs (seulement 28 bits utilisés réellement)
2. Étape 2: **Connaître la taille d'un bloc** (souvent donnée dans):
 - Exemple : 512 octets, 4 Ko, 32 Ko...
 - $32\text{Ko}=32\times 1024=32768$ octets par bloc.
3. Étape 3 : **Multiplier et Convertir :**
Multiplier : Nombre de bloc * Taille du bloc = Capacité maximal
Convertir : en Ko, Mo, Go ou To pour exprimer le résultat clairement.

Exercice 2 : Indexation par inodes (Taille du plus grand fichier)

La figure représente une vue simplifiée d'un système de fichiers de type UNIX utilisant des inodes pour gérer les fichiers et les répertoires. Le chemin accédé est `/home/pierre/courrier`.

On considère que :

- La taille d'un bloc est de **1 Ko**,
- Une adresse de bloc occupe **4 octets**,
- L'inode contient **10 pointeurs directs** (0 à 9), **1 pointeur indirect simple** (bloc 10), **1 pointeur indirect double** (bloc 11) et **1 pointeur indirect triple** (bloc 12).

Q1. Expliquez comment le système accède au fichier `courrier` en partant du répertoire racine `/`.

Correction Q1 : Le répertoire racine (`/`) pointe vers l'inode 6. Dans ce répertoire (bloc 132), l'entrée `home` est associée à l'inode 147. Le contenu de l'inode 147 (bloc 392) contient l'entrée `pierre`, qui est associée à l'inode 392. Le fichier `courrier` est trouvé dans le contenu de l'inode 202 (dans le bloc de répertoire), qui référence alors un inode contenant les blocs de données du fichier `courrier`.

(1)/home(6)/pierre(147)/courrier(202)/.

Q2. Combien d'accès disques sont nécessaires pour accéder à un bloc de données pointé par :

- a) Un pointeur direct ?
- b) Un pointeur indirect simple ?
- c) Un pointeur indirect double ?
- d) Un pointeur indirect triple ?

Correction Q2 :

Chaque bloc d'indirection peut donc contenir :

1024 octets par bloc / 4 octets par adresse = 256 adresses

- a) Direct : 1 (inode) + 1 (bloc de données) = **2 accès**
- b) Simple indirect : inode + bloc d'adresses + bloc de données = **3 accès**
- c) Double indirect : inode + 1er bloc + 2e bloc + données = **4 accès**
- d) Triple indirect : inode + bloc1 + bloc2 + bloc3 + données = **5 accès**

Q3. Calculez la taille maximale que peut atteindre un fichier si le système impose une limite de **4 accès disques** pour accéder à n'importe quel octet. (Précisez votre raisonnement et les types d'indirection utilisables.)

Correction Q3 : Types de pointeurs autorisés :

- Directs : 10 blocs
- Indirect simple : 256 blocs
- Indirect double : $256 \times 256 = 65536$ blocs Total = $10 + 256 + 65536 = \mathbf{65792 \text{ blocs}}$
Taille max = 65792 Ko = **~64.25 Mo**

Q4. Quelle serait la taille maximale si aucune contrainte de nombre d'accès disque n'était imposée ?

Correction Q4 : On peut utiliser la triple indirection :

- Triple indirect = $256 \times 256 \times 256 = 16,777,216$ blocs Total = $10 + 256 + 65536 + 16,777,216 = \mathbf{16,843,018 \text{ blocs}}$ Taille max = 16,843,018 Ko = **~16 Go**

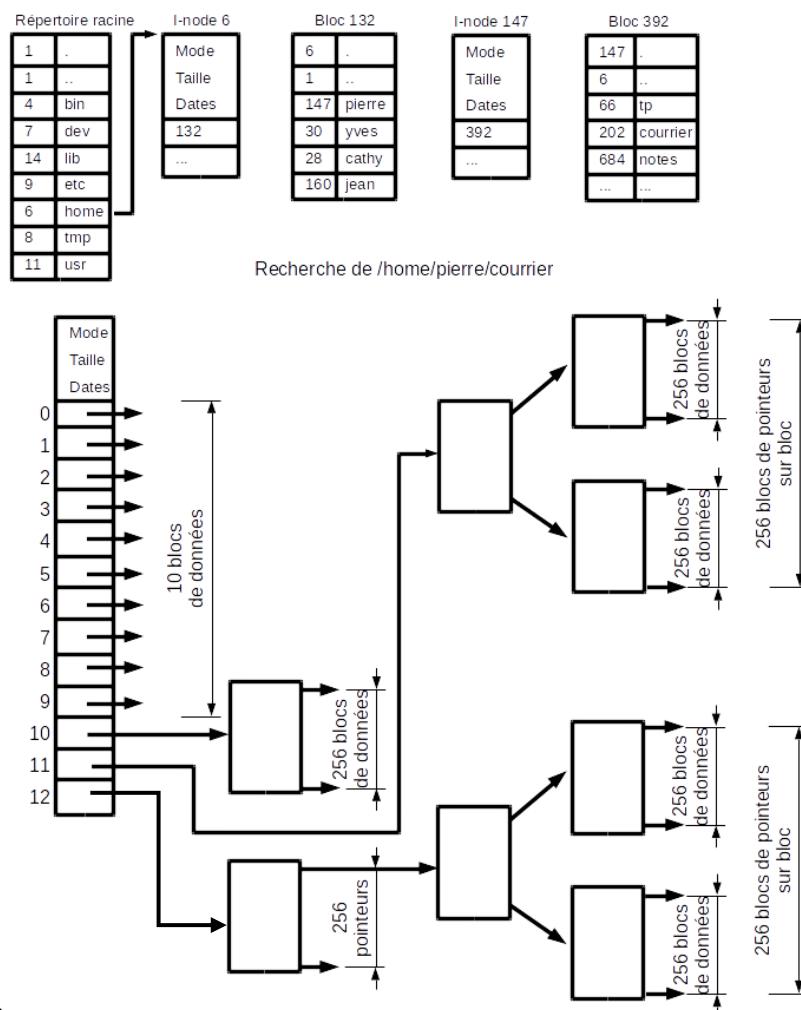


Figure 2 – i-nodes